TOP 10 PYSPARK CONCEPTS EVERY DATA ENGINEER MUST KNOW

BY MICHAEL FERNANDES

1. SPARKSESSION

SparkSession is the starting point of every PySpark application. It combines the functionality of older APIs such as SQLContext, HiveContext, and SparkContext.

Without SparkSession, you cannot:

- Create DataFrames
- Read/write files
- Execute SQL queries
- Access Spark configurations

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
 .appName("EmployeeAnalysis") \
 .getOrCreate()
```

2. DATAFRAMES

A distributed table-like structure organized into rows and columns.

Why DataFrames instead of RDDs?

- Catalyst Query Optimizer
- Better memory management
- Faster execution
- SQL support

```
df = spark.read.csv("employees.csv", header=True)

df.show()

#Common Operations

df.select("name")
df.filter(df.salary > 50000)
df.withColumn("bonus", df.salary * 0.1)
```

3. TRANSFORMATIONS VS ACTIONS

Transformations - Create a new DataFrame/RDD.

Actions are used to Trigger execution.

```
# Transformations

filter()
select()
join()
groupBy()
withColumn()


# Actions

show()
count()
collect()
take()
write()
```

4. LAZY EVALUATION

Lazy Evaluation means Spark does not execute transformations immediately. Instead, it records the sequence of operations and waits until an action is called.

Step 1: Read Data

```
df = spark.read.csv("employees.csv",header=True)
```

Step 2: Apply Transformations and Actions

```
filtered_df = df.filter(df.salary > 50000)
selected_df = filtered_df.select("name")
sorted_df = selected_df.orderBy("name")
# Stlll no Execution

sorted_df.show()
# Now Spark executes the entire workflow.
```

5. RDD (RESILIENT DISTRIBUTED DATASET)

The original low-level distributed data structure in Spark.

Features

- Immutable
- Distributed
- Fault tolerant

Use RDD when:

- Need fine-grained control
- Working with unstructured data
- Custom distributed logic


```
rdd = spark.sparkContext.parallelize([1,2,3,4]

rdd.map(lambda x: x*2).collect()
```

6. PARTITIONING

A partition is a chunk of data processed by one executor task.

Performance depends heavily on partitioning.
Too few partitions:

- Poor parallelism

Too many partitions:

- Scheduling overhead

```
# Repartition - Full shuffle.
df.repartition(10)

# Coalesce - Reduce Partitions with Minimal Shuffle
df.coalesce(5)
```

7. JOINS

A join combines data from two or more DataFrames based on a common column (key). PySpark provides the same SQL functionality on distributed data.

```
# Inner Join
df1.join(df2, "id", "inner")

# Left Join
df1.join(df2, "id", "inner")

# Right Join
df1.join(df2, "id", "inner")

# Full Join
df1.join(df2, "id", "inner")
```

8. WINDOW FUNCTIONS

Perform calculations across related rows without collapsing data.

rank()
dense_rank()
row_number()
lead()
lag()
sum()
avg()

```
from pyspark.sql.window import Window
from pyspark.sql.functions import rank

window_spec = Window.partitionBy("Dept") \
                    .orderBy("Salary")
df.withColumn(
    "rank",
    rank().over(window_spec))
```

9. SPARK SQL & CATALYST OPTIMIZER

Spark SQL - Allows querying DataFrames using SQL.

Catalyst Optimizer - Spark's query optimization engine.
It automatically:

- Reorders operations
- Pushes filters down
- Removes unnecessary computations
- Optimizes joins

```
df.filter(df.salary > 50000)
# Catalyst converts it into an optimized physical execution plan.

#Check Plan
df.explain(True)
```

10. CACHING AND PERSISTANCE

Caching stores the computed DataFrame in memory so Spark can reuse it instead of recomputing.

Persistence is a more flexible version of caching. It allows you to choose where Spark stores the data.

```
# Catching
df_filtered.cache()

# Persisting
df.persist()
```

THANK YOU

BY MICHAEL FERNANDES